Atelier pratique n°3 : Les bases de Git

Correction complète — commandes, résultats attendus et réponses aux questions

Partie 1 — Configuration et création du projet

Commandes :

```
git --version
git config --global user.name "Mariam Tahayekt"
git config --global user.email "votre@email.com"
mkdir atelier-git
cd atelier-git
git init
git status
```

Résultats attendus :

- `git --version` affiche par exemple `git version 2.43.0`. Si la commande n'est pas reconnue, il faut installer Git (git-scm.com).
- `git init` affiche `Initialized empty Git repository in .../atelier-git/.git/`
- `git status` affiche `On branch main / No commits yet / nothing to commit`.

Q1. À quoi sert la commande git init ?

Elle initialise un nouveau dépôt Git dans le dossier courant. Elle crée le dossier caché `.git`, qui contient toute la base de données du projet (historique, configuration, branches). Sans elle, Git ne suit rien.

Q2. Quel est le rôle des commandes git config ?

Elles définissent la configuration de Git. Ici, `user.name` et `user.email` enregistrent l'identité de l'auteur, qui sera associée à chaque commit. L'option `--global` applique ce réglage à tous les dépôts de l'utilisateur sur la machine.

Q3. Que signifie « working directory » ?

C'est le répertoire de travail : le dossier du projet tel qu'il apparaît sur le disque, avec les fichiers que l'on modifie directement. Les changements y sont d'abord « non suivis » ou « non indexés », puis passent au staging (`git add`) et enfin au dépôt (`git commit`).

Partie 2 — Suivi des fichiers et premier commit

Création du fichier (au choix : éditeur de texte, ou ligne de commande) :

```
echo "<h1>Mon premier projet Git</h1>" > index.html
```

Commandes :

```
git status
git add index.html
git status
git commit -m "Initialisation du projet"
git log
```

Résultats attendus :

- Premier `git status` : `Untracked files: index.html` (fichier non suivi).
- Après `git add` : `Changes to be committed: new file: index.html`.

- Après le commit : `[main (root-commit) a1b2c3d] Initialisation du projet`.
- `git log` affiche le hash complet du commit, l'auteur, la date et le message.

Q1. Différence entre git add et git commit ?

`git add` place les modifications choisies dans la zone de staging (préparation). `git commit` enregistre définitivement ce qui est dans le staging sous forme d'une version dans l'historique, avec un message.

Q2. Rôle de la zone de staging ?

C'est une zone intermédiaire entre le working directory et le dépôt. Elle permet de choisir précisément quelles modifications feront partie du prochain commit, et donc de créer des commits propres et cohérents, même si plusieurs fichiers ont été modifiés.

Q3. À quoi sert l'option -m ?

Elle permet d'écrire le message du commit directement dans la commande. Sans `-m`, Git ouvre un éditeur de texte pour saisir le message.

Partie 3 — Modifier un fichier

Modification de index.html (ajout d'un paragraphe) :

```
echo "<p>Bonjour, je suis Mariam, etudiante en developpement digital.</p>" >> index.html
```

Commandes :

```
git status
git diff
git add index.html
git diff --staged
git commit -m "Ajout de la présentation"
git log --oneline
```

Résultats attendus :

- `git status` : `Changes not staged for commit: modified: index.html`.
- `git diff` : la ligne ajoutée apparaît précédée de + (en vert), les lignes supprimées par - (en rouge).
- Après `git add`, `git diff` ne montre plus rien, mais `git diff --staged` affiche la modification.
- `git log --oneline` affiche 2 lignes (une par commit) : hash court + message.

Q1. Que montre git diff ?

Il montre les différences ligne par ligne entre le working directory et la zone de staging : c'est-à-dire les modifications faites mais pas encore ajoutées au staging.

Q2. Différence entre git diff et git diff --staged ?

`git diff` compare le working directory au staging (modifications non indexées). `git diff --staged` compare le staging au dernier commit (modifications prêtes à être commitées).

Q3. Pourquoi créer plusieurs commits au lieu d'un seul ?

Pour garder un historique clair et exploitable : chaque commit représente une étape logique avec un message explicite. Cela facilite la relecture, la recherche d'un bug, le retour à une version précédente, l'annulation d'un changement précis et le travail en équipe.

Partie 4 — Gérer plusieurs fichiers

Création des fichiers :

```
echo "h1 { color: #4f46e5; }" > style.css
echo "# Atelier Git" > README.md
echo "Projet d'initiation aux bases de Git." >> README.md
```

Commandes :

```
git status
git add style.css README.md
git status
git commit -m "Ajout du style et de la documentation"
git log --oneline
```

Résultats attendus :

- Premier `git status` : `Untracked files: README.md, style.css`.
- Après `git add` : les deux fichiers sont sous `Changes to be committed`.
- `git log --oneline` affiche maintenant 3 commits, le plus récent en haut :

```
c3d4e5f Ajout du style et de la documentation
b2c3d4e Ajout de la présentation
a1b2c3d Initialisation du projet
```

(Les hash seront différents chez vous.)

Q1. Différence entre fichier tracked et untracked ?

Un fichier **tracked** (suivi) est connu de Git : il figure dans le dernier commit ou dans le staging, et ses modifications sont détectées. Un fichier **untracked** (non suivi) existe dans le dossier mais Git ne le connaît pas encore ; il faut faire `git add` pour commencer à le suivre.

Q2. Peut-on ajouter plusieurs fichiers avec un seul git add ?

Oui. On peut lister plusieurs fichiers (`git add style.css README.md`), ou tout ajouter d'un coup avec `git add .` (ou `git add -A`).

Q3. À quoi sert le fichier README.md ?

C'est le fichier de présentation du projet, écrit en Markdown. Il explique ce qu'est le projet, comment l'installer et l'utiliser. Il s'affiche automatiquement sur la page d'accueil d'un dépôt GitHub ou GitLab.

Partie 5 — Bilan et vérification

Commande	Rôle
`git init`	Initialise un nouveau dépôt Git dans le dossier courant (crée le dossier .git).
`git status`	Affiche l'état du dépôt : fichiers non suivis, modifiés, ou prêts à être commités.
`git add`	Ajoute des fichiers ou modifications à la zone de staging, en préparation du commit.
`git commit`	Enregistre les changements du staging dans l'historique sous forme d'une version avec un message.
`git log`	Affiche l'historique des commits (hash, auteur, date, message). Avec `--oneline`, une ligne par commit.
`git diff`	Montre les différences entre les versions (working directory / staging / dernier commit).

Vérification du livrable

- Dossier `atelier-git` contenant **index.html**, **style.css** et **README.md**.
- Dépôt local initialisé (dossier caché `.git` présent).
- Au moins 3 commits avec messages explicites : Initialisation du projet, Ajout de la présentation, Ajout du style et de la documentation.
- **Capture d'écran** de `git log --oneline` : à faire vous-même sur votre machine.